

KITCommunity

Comprehensive Platform Blueprint

An enterprise-grade digital campus platform unifying academic communication, departmental discussions, extracurricular clubs, placement drives, and peer-to-faculty interaction into one performant web and mobile application for the KIT ecosystem.

React 18 + Vite

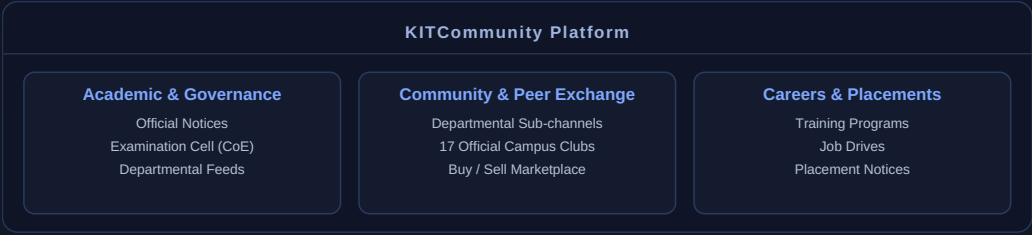
Node.js + Express

MongoDB + Mongoose

JWT Dual-Token Auth

Capacitor Android

Google OAuth 2.0

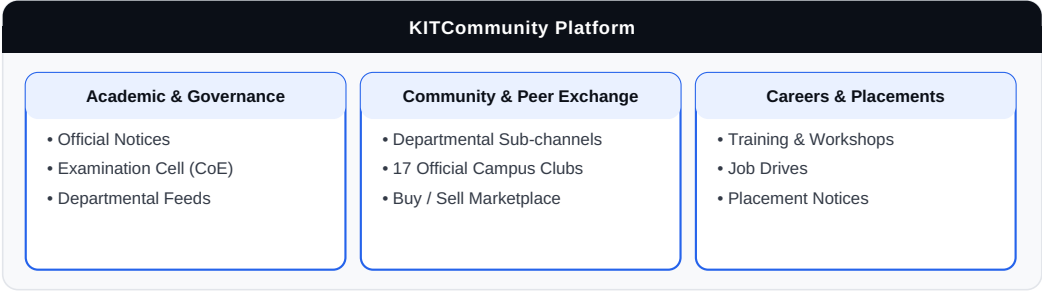


Executive Summary & Project Overview

KITCommunity is an enterprise-grade digital campus platform designed specifically for the higher education ecosystem at KIT. The platform unifies academic communication, departmental discussions, extracurricular club activities, placement drives, and direct peer-to-faculty interaction into a single performant web and mobile application.

Architecturally, KITCommunity combines the structured, professional readability of **LinkedIn** with the community-driven engagement mechanics of **Reddit** — dynamic thread ranking, reputation scores, sub-channel groups, and nested discussions.

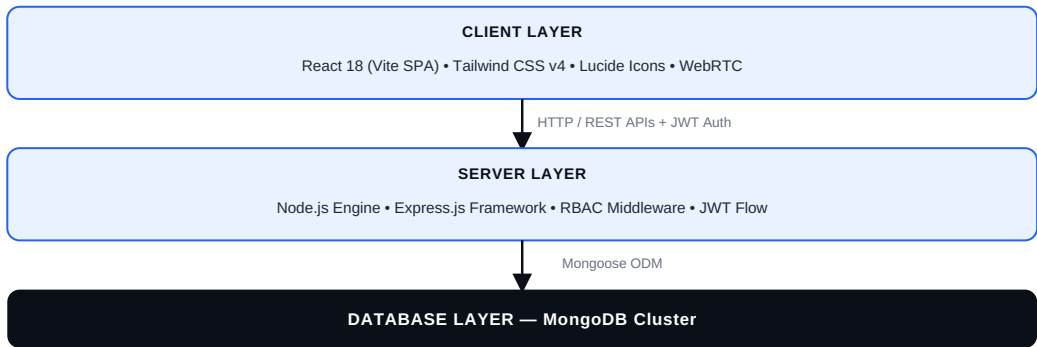
Platform Pillars



KITCommunity positions itself as the connective social layer of a broader institutional technology stack, designed to scale from single-campus deployment toward the multi-service CampusOS ecosystem outlined in Section 6.

Platform Architecture & Core Engineering Capabilities

Core Technology Stack



Frontend: React 18 initialized with Vite for rapid Hot Module Replacement (HMR). Styled using Tailwind CSS v4 coupled with native CSS Design Tokens. Vector visuals use standard Lucide React icon motifs.

Backend: Node.js and Express.js REST API layer utilizing JSON Web Token (JWT) dual-token mechanics (short-lived access tokens + long-lived refresh tokens) and `bcryptjs` password hashing.

Database: MongoDB with Mongoose Object Data Modeling (ODM).

Mongoose Database Schemas

MODEL	RESPONSIBILITY
User	Manages user profiles, role-based access control (RBAC: student, faculty, moderator, admin), reputation scores, departmental affiliation, and bookmarked posts (<code>savedPosts</code>).
Channel	Encapsulates category groupings, channel types (general, branch, interest), and designated SVG motif identifiers.
Post	Stores titles, rich-text bodies, tags, media attachments (images, videos, PDFs), voting arrays, aggregated scores, calculated <code>hotScore</code> , and official status flags (<code>isNotice</code>).
Comment	Implements an adjacency list pattern supporting up to 3 tiers of nested discussion threads.
Notification	Handles real-time events (upvotes, comments, mentions) with read / unread flags.
Message	Powers 1-on-1 private communications between students, faculty, and administrators.

UI/UX System & Channel Taxonomy

Visual Theme Tokens

Outer Canvas #F4F2EE	Content Container #FFFFFF	Brand Accent #2563EB	Dark Mode Base #0B0F17
--	---	--	--

Icon Motif Framework

Standardized SVG icons map directly to channels to eliminate visual noise.

Megaphone	BookOpen	Briefcase	Code	Cpu	FlaskConical	Building2	Radio	Zap	Wrench	Terminal	Sparkles	Trophy	Award
Calendar	FileText	HelpCircle											

Channel Taxonomy & Information Architecture

Channels are categorized into six structural groups:

KITCommunity Information Hierarchy

1. Official Announcements

#official-notices #general-lounge

2. Communities (Reddit-Style)

r/Tech_Discussions r/Research_Innovations r/Startup_Ideas r/Alumni_Network r/Gaming_eSports r/Buy_Sell_Trade

3. Examination Cell & CoE

#exam-schedules #hall-tickets #exam-results #exam-inquiries

4. Engineering Departments

CSE AIML Biotech Civil ENTC Electrical Mechanical

5. Careers & Placements

#placement-drives #training-and-workshops #job-opportunities-internships #career-notices

6. Official Campus Clubs (17 Organizations)

ISTE E-Cell KITCoEK Team Mavericks SDC Writers Club EBSB Cultural Club AURA Shourya Women Cell Rotaract SWE Walk With World NCC NSS Lead India Petrichor Green Club

Algorithmic Ranking, Reputation & Interactivity

Reddit-Style Logarithmic Hot Score Algorithm

To prevent older, highly-voted posts from permanently dominating feeds, KITCommunity implements a logarithmic decaying hot ranking algorithm:

$$\text{sign}(S) = \begin{cases} 1 & \text{if } S > 0 \\ 0 & \text{if } S = 0 \\ -1 & \text{if } S < 0 \end{cases}$$

$$\text{Score} = \text{Upvotes} - \text{Downvotes}$$

$$\text{HotScore} = \log_{10}(\max(1, |S|)) + \frac{\text{sign}(S) \cdot (t - t_0)}{45000}$$

Where t represents the post creation timestamp, t_0 is a fixed base epoch, and 45000 scales time decay (roughly 12.5 hours per point magnitude).

Reputation System Matrix

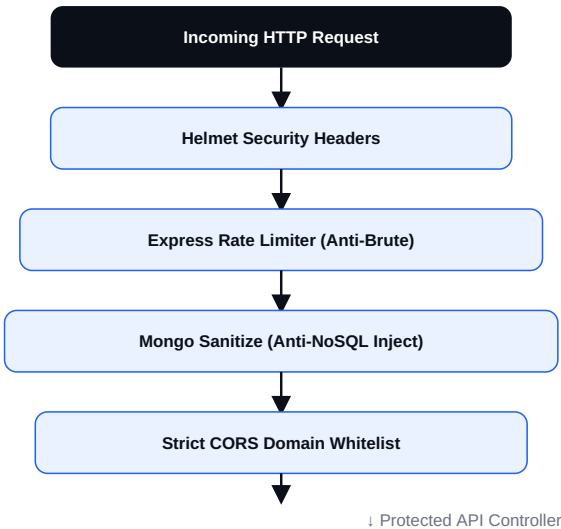
Action Event	Point Adjustment	Algorithmic Impact
Post Upvoted	+5 points	Boosts user profile standing and author weight.
Post Downvoted	-2 points	Reduces user profile standing.
Comment Added	+2 points	Encourages constructive participation.
Self-Voting Attempt	0 points (Blocked)	Prevented on both client and backend layers.

Interactivity, Communications & Media Processing

- **AST Text Parsing:** Automatic regular expression parsing converts `@username` mentions and `#hashtag` entities into interactive router links.
- **Multi-Format File Processing:** Built-in support for direct image rendering, native video playback controls, and downloadable inline PDF documents.
- **KITMeet WebRTC Teleconferencing:** Integrated video call modal embedded in the primary navigation bar, powered by WebRTC / Jitsi Meet for instant campus meetings and remote lectures.
- **Global Dynamic Search:** Client-side and server-side indexing allowing live keyword filtering across post titles, bodies, tags, and user handles.

Security Audit & Hardening Strategy

To ensure data integrity and compliance prior to institution-wide deployment, the following security controls must be applied:

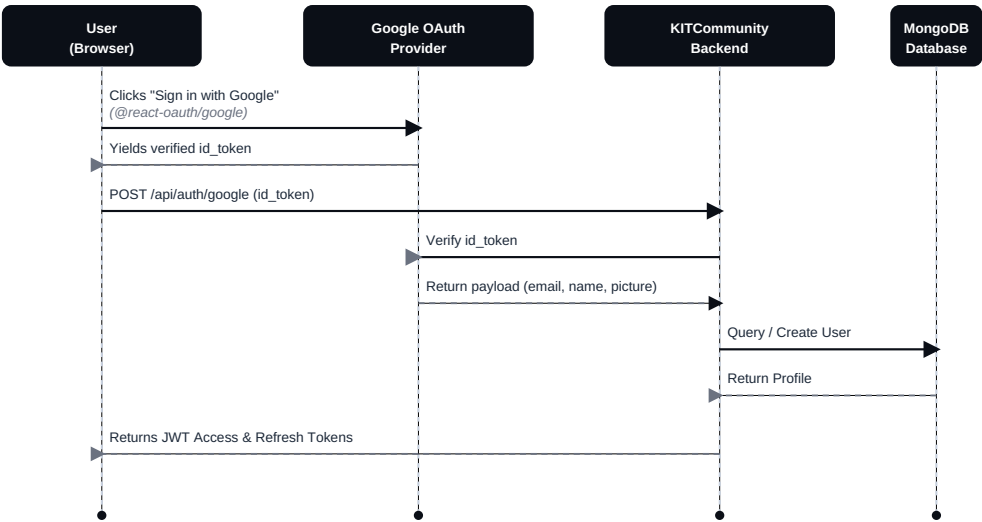


SECURITY VECTOR	RISK	MITIGATION PLAN	IMPLEMENTATION DETAIL
Brute-Force Attacks	High	Rate limiting on authentication routes.	Apply <code>express-rate-limit</code> , max 10 requests / 15-minute window on <code>/api/auth/login</code> .
NoSQL Injection	High	Sanitize user inputs to strip MongoDB operators.	Intercept <code>req.body</code> , <code>req.query</code> , <code>req.params</code> with <code>mongo-sanitize</code> to strip <code>\$</code> and <code>.</code> .
HTTP Security Headers	Medium	Standardize browser security controls.	Configure <code>helmet</code> to enforce strict HSTS, CSP, and X-Frame-Options.
Cross-Origin Exposure	Medium	Restrict resource sharing to approved domains.	Configure <code>cors</code> with an explicit domain whitelist (<code>kitcommunity.kit.edu</code>).
Credential & Key Leaks	Medium	Prevent secret exposure through key rotation.	Store cryptographic keys in environment variables; automate JWT secret rotation.

Federated Identity: Google OAuth 2.0 Integration

To simplify user onboarding, KITCommunity supports single sign-on (SSO) restricted to institutional domain accounts (@kit.edu).

OAuth 2.0 Authentication Flow



Solid arrows represent client-initiated or server-initiated requests; dashed arrows represent the corresponding responses returned back up the chain.

Step-by-Step Implementation Protocol

- 1 **Google Cloud Console Configuration** — create a project titled `KITCommunity-Auth`, configure the OAuth Consent Screen, and create an OAuth 2.0 Client ID (Web Application). Add `kitcommunity.kit.edu` to Authorized JavaScript Origins and `/api/auth/google/callback` to Authorized Redirect URIs.
- 2 **Client-Side Integration** — install and wrap the app with `GoogleOAuthProvider`.
- 3 **Server-Side Integration** — verify the ID token and enforce the domain restriction before issuing tokens.

Client-Side (client/)

```
npm install @react-oauth/google
```

```
import { GoogleOAuthProvider, GoogleLogin } from '@react-oauth/google';

<GoogleOAuthProvider clientId={import.meta.env.VITE_GOOGLE_CLIENT_ID}>
  <GoogleLogin
    onSuccess={handleGoogleSuccess}
    onError={() => console.error('Google Login Failed')}
  />
</GoogleOAuthProvider>
```

Server-Side (server/) — authController.js

```
npm install google-auth-library
```

```
const { OAuth2Client } = require('google-auth-library');
const client = new OAuth2Client(process.env.GOOGLE_CLIENT_ID);

exports.googleSignIn = async (req, res) => {
  const { id_token } = req.body;
  const ticket = await client.verifyIdToken({
    idToken: id_token,
    audience: process.env.GOOGLE_CLIENT_ID,
  });
  const { email, name, picture } = ticket.getPayload();

  if (!email.endsWith('@kit.edu')) {
    return res.status(403).json({ error: 'Access restricted to official @kit.edu domain emails.' });
  }

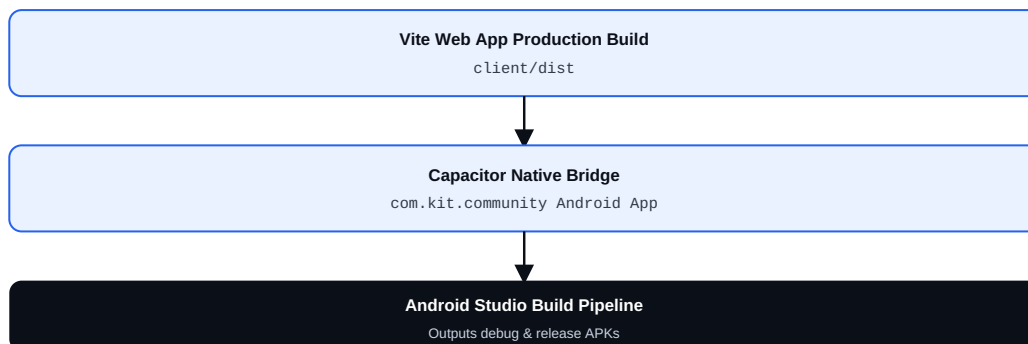
  let user = await User.findOne({ email });
  if (!user) {
    user = await User.create({ email, name, avatar: picture, role: 'student' });
  }

  const accessToken = generateAccessToken(user);
  const refreshToken = generateRefreshToken(user);

  res.json({ accessToken, refreshToken, user });
};
```

Mobile Deployment: Native Android APK Generation

KITCommunity uses **Capacitor** to convert the Vite React web application into a native Android bundle without rewriting code.



Build Execution Sequence

- 1 Install Capacitor Dependencies inside `/client`:

```
cd client
npm install @capacitor/core @capacitor/cli @capacitor/android
```

- 2 Initialize Capacitor Configuration, pointing to the production build output directory:

```
npx cap init KITCommunity com.kit.community --web-dir dist
```

- 3 Generate Web Build Assets by compiling the React bundle:

```
npm run build
```

- 4 Add & Synchronize Android Platform, injecting the native container and copying assets:

```
npx cap add android
npx cap copy android
```

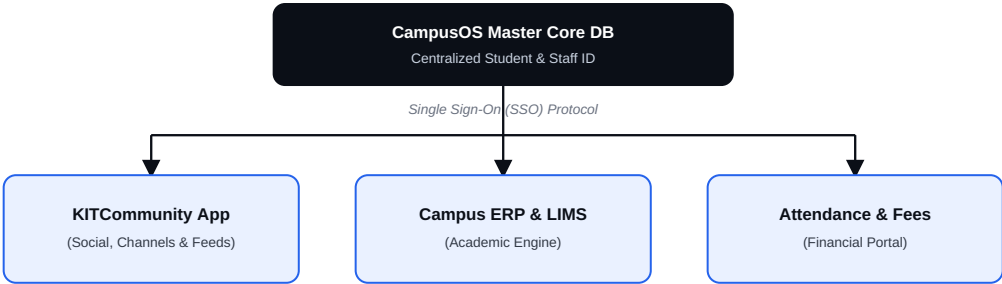
- 5 Android Studio Compilation & Signing — launch with `npx cap open android`, allow Gradle sync, then navigate to **Build > Build Bundle(s) / APK(s) > Build APK(s)**. Output path:

```
client/android/app/build/outputs/apk/debug/app-debug.apk
```

Institutional Integration: CampusOS Master Database Sync

As the institution expands its digital services, KITCommunity will serve as the social and communication layer of the broader **CampusOS** platform.

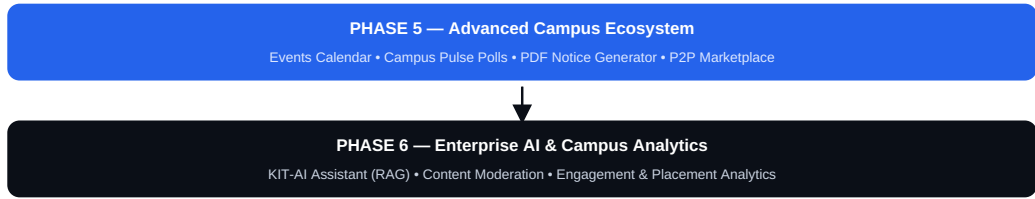
Enterprise Microservice Architecture



Key Integration Workflows

- 1 **Master Identity Mapping:** The `User` model includes an explicit foreign reference field: `campusos_user_id` (indexed, unique).
- 2 **Unified Session Token Exchange:** Authenticating via CampusOS yields a master token that KITCommunity validates asynchronously, enabling smooth single sign-on across campus tools.
- 3 **Event-Driven Webhook Pipeline:** Actions in external subsystems automatically publish corresponding announcements in KITCommunity — e.g. publishing exam schedules in the ERP triggers a webhook payload that creates an official notice in `#exam-schedules`.

Future Strategic Expansion Roadmap



Phase 5 — Advanced Campus Ecosystem

1. Interactive Campus Events & Hackathons Calendar

Central interface listing tech fests, workshops, hackathons, and placement drives. Interactive RSVP (*Attending / Interested / Not Attending*), `.ics` calendar export for Google/Outlook/Apple Calendar, venue mapping, and automated registration tracking.

2. Interactive Campus Pulse Polls

Dynamic polling widgets attachable to posts by Faculty, Admins, or Student Council. Role-gated voting, live animated results, and database-level anti-tamper unique-vote constraints per user.

3. Official PDF Notice Generator for Faculty

Instant formatted institutional notices with digital letterhead and watermark seals, rendered client-side via `jspdf` and `html2canvas`, including QR codes linking back to the official post for authenticity verification.

4. Verified Peer-to-Peer Campus Marketplace

Dedicated `r/Buy_Sell_Trade` marketplace for academic materials with structured listings, Verified Student trust badges tied to `@kit.edu` auth state and reputation score, and moderation workflows for flagging unauthorized commercial items.

Phase 6 & Immediate Execution Plan

Phase 6 — Enterprise AI & Campus Analytics

1. KIT-AI Student Assistant Bot

Embedded AI chatbot answering common inquiries on academic policies, exam rules, and campus services. Uses Retrieval-Augmented Generation (RAG) connecting an LLM to a vector database of institutional handbooks, syllabi, and FAQs, with contextual routing that escalates low-confidence queries to departmental staff via private DM.

2. Automated Content Moderation & Sentiment Analysis

Automated moderation layer scanning proposed posts and comments for inappropriate language, hate speech, or harassment before publication, with an automated action engine that flags violations for human review and applies temporary rate restrictions on repeat offenders.

3. Department Engagement & Placement Analytics Dashboard

Executive dashboard with engagement heatmaps across departments, trending technology tag analysis (`#react` , `#python` , `#cad`) to inform training programs, and placement metrics tracking notice engagement, event attendance, and drive outcomes.

Immediate Action Item Matrix

DOMAIN	OPERATIONAL TASK	STATUS
Mobile Build	Install <code>@capacitor/core</code> and compile the initial Android APK.	Pending
Identity / Auth	Set up the Google OAuth Client ID and enforce the <code>@kit.edu</code> domain restriction.	Pending
Security	Inject <code>helmet</code> , rate limiting, and <code>mongo-sanitize</code> middleware.	Pending
Database	Update the Mongoose <code>User</code> schema with an indexed <code>campusos_user_id</code> field.	Pending